



ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЦИФРОВОГО ПРОЕКТИРОВАНИЯ

БГУИР

ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ СИСТЕМ И СЕТЕЙ

КАФЕДРА ПОИТ

ВЕСНА 2026



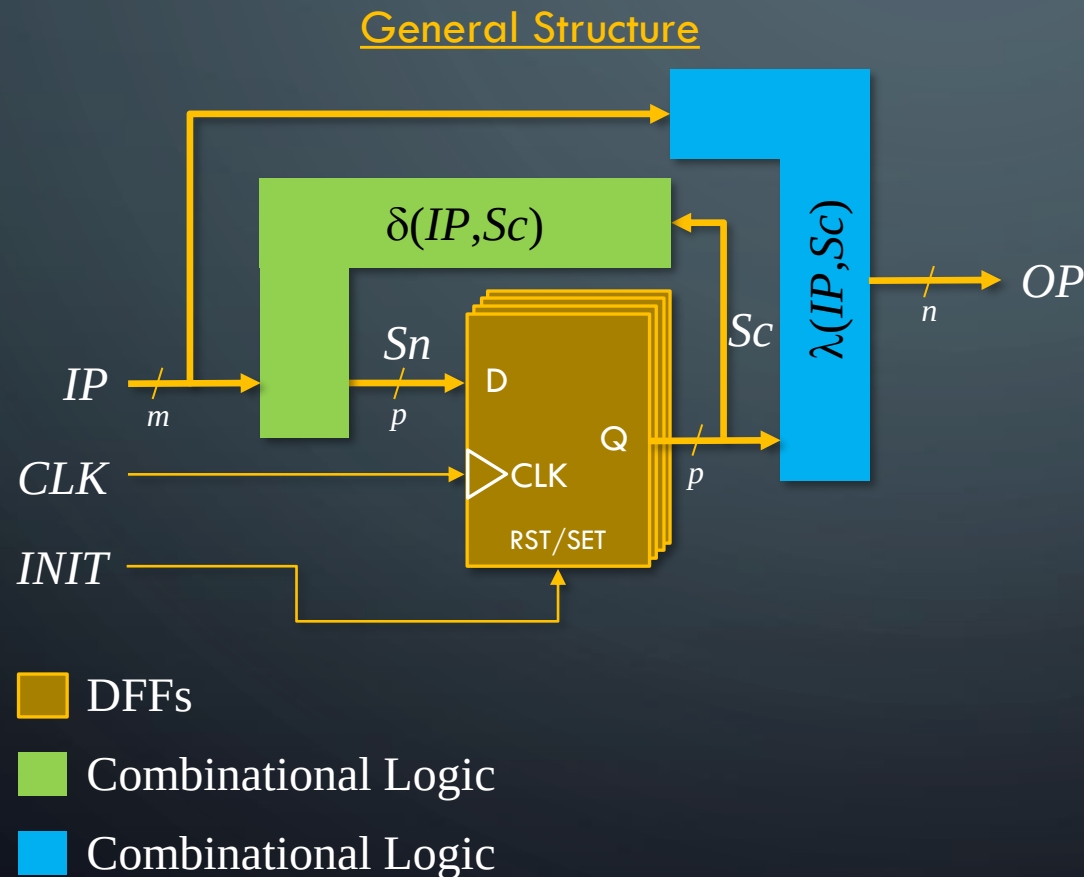
ИВАНЮК А.А.

ПОЦП :: Л9 :: Язык VHDL :: последовательные схемы :: FSM

A digital **Finite State Machine (FSM)** is a sequential logic circuit that models system behavior using a limited set of states, transitions, inputs, and outputs.

Only **synchronous** FSMs will be considered.

FSM



$$FSM = \{S, S_0, I, O, \delta, \lambda\}$$

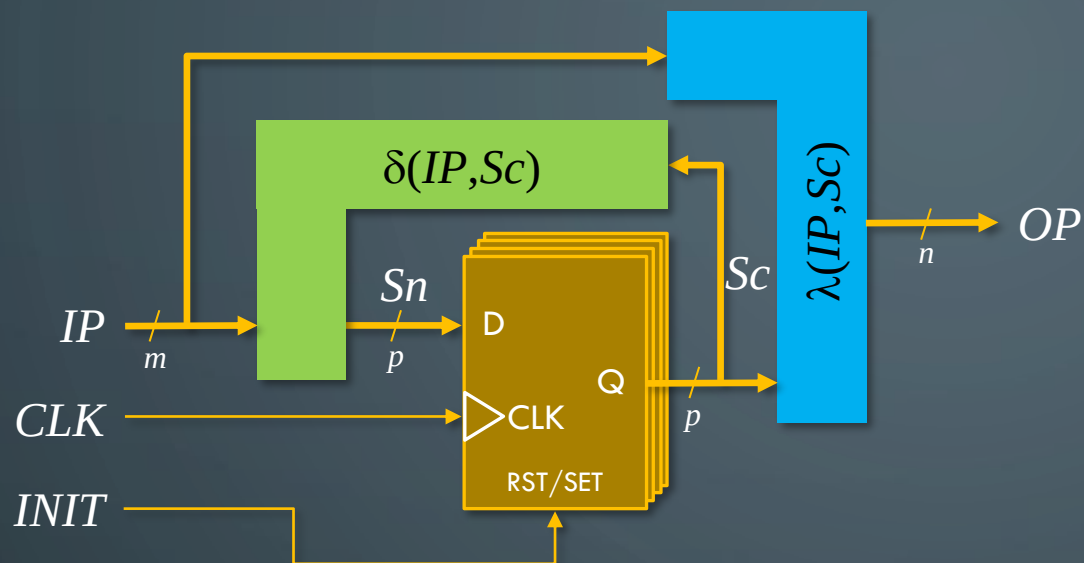
- S – a set of states ($\max|S| = 2^p$)
- S_0 – an initial state
- I – input alphabet of m -bit symbols
- O – output alphabet of n -bit symbols
- δ – the next state function
- λ – the output function

- Sc – the current state
- Sn – the next state
- $Sn = \delta(IP, Sc)$ with combinational delay
- $Sc = Sn$ on edge of CLK

Types of FSM

ПОЦП :: Л9 :: Язык VHDL :: последовательностные схемы :: FSM

Mealy FSM

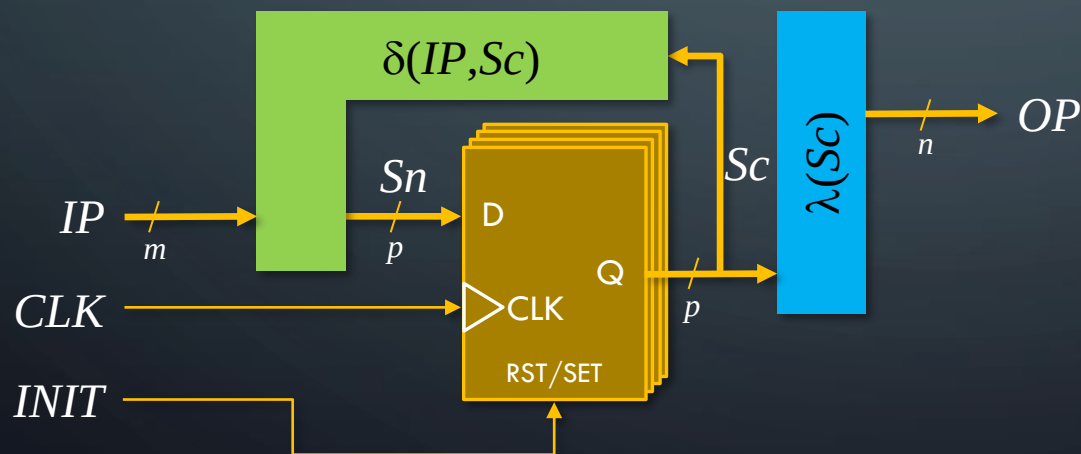


Output symbol depends on
Current State and Input Symbol:
 $OP = \lambda(IP, Sc)$

Next State depends on
Current State and Input Symbol:

$$Sn = \delta(IP, Sc)$$

Moore FSM



Output symbol depends only on
Current State:

$$OP = \lambda(Sc)$$

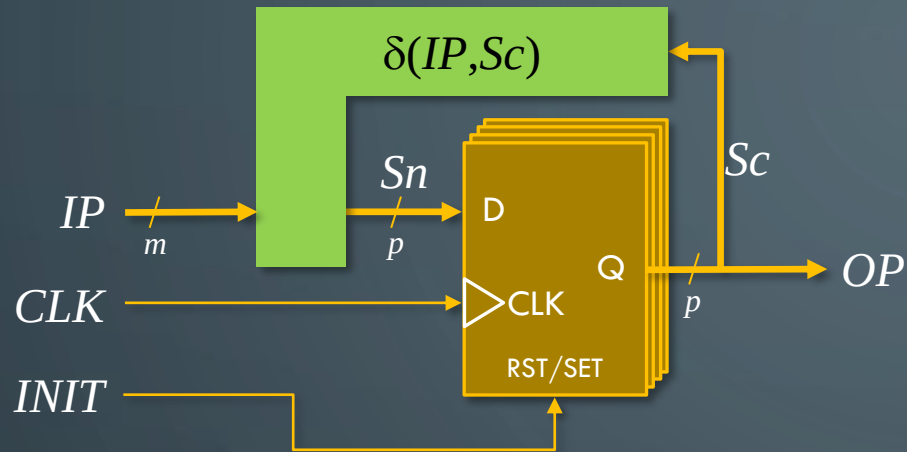
Next State depends on
Current State and Input Symbol:

$$Sn = \delta(IP, Sc)$$

Types of FSM

ПОЦП :: Л9 :: Язык VHDL :: последовательностные схемы :: FSM

Medvedev FSM

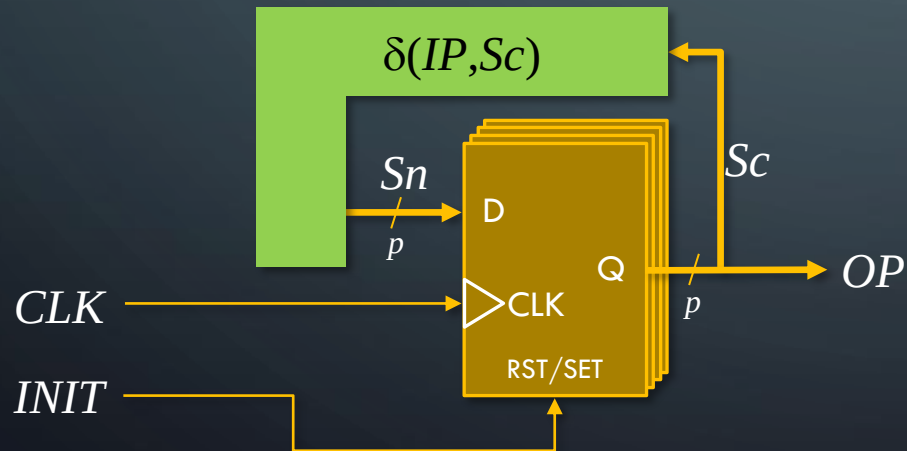


Output symbol equals to
Current State:
 $OP = Sc$

Next State depends on
Current State and Input Symbol:

$$Sn = \delta(IP, Sc)$$

Autonomous FSM



Output symbol equals to
Current State:
 $OP = Sc$

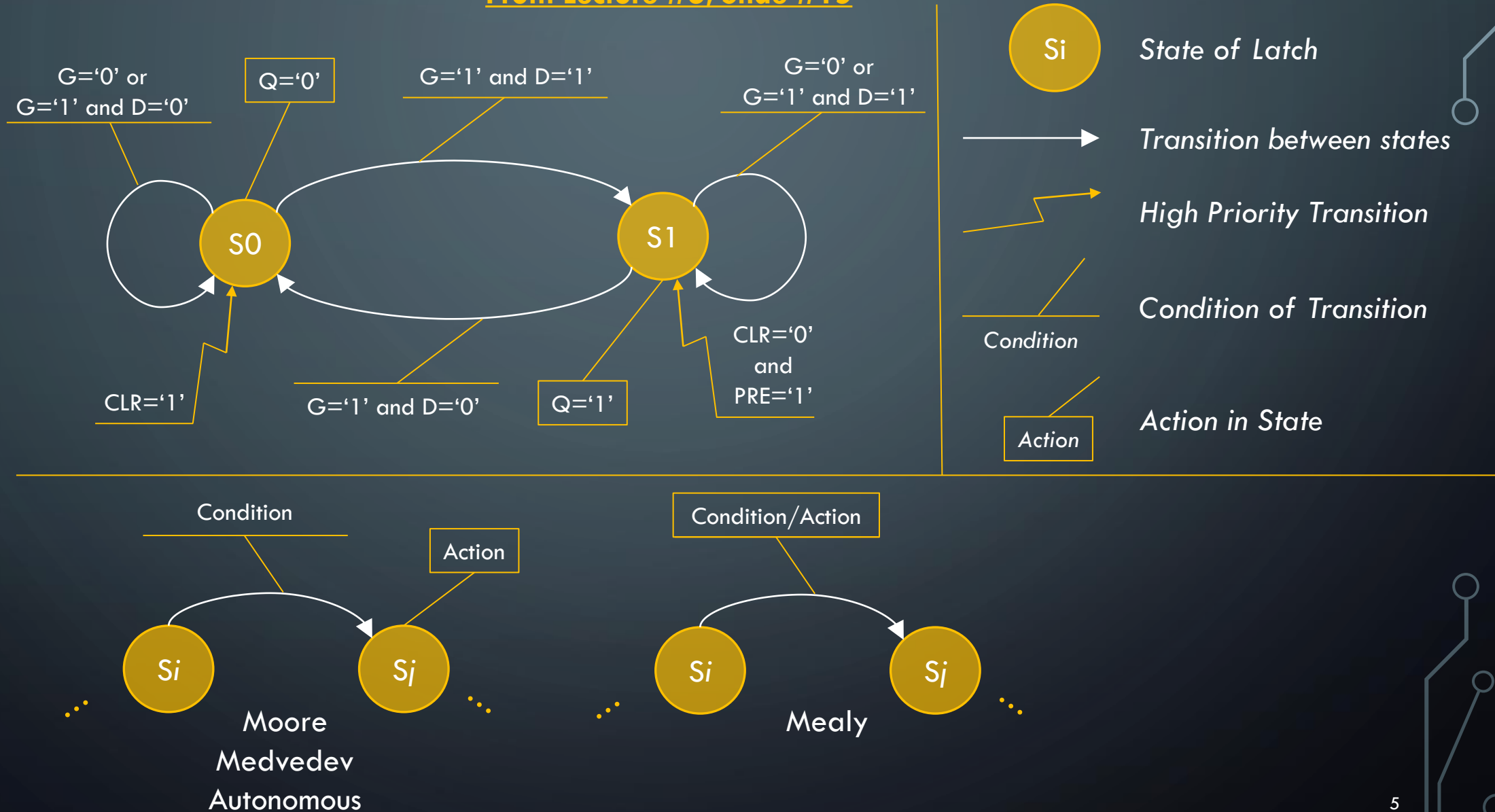
Next State depends only on
Current State:

$$Sn = \delta(Sc)$$

ПОЦП :: Л9 :: Язык VHDL :: последовательностные схемы :: FSM

From Lecture #6, Slide #15

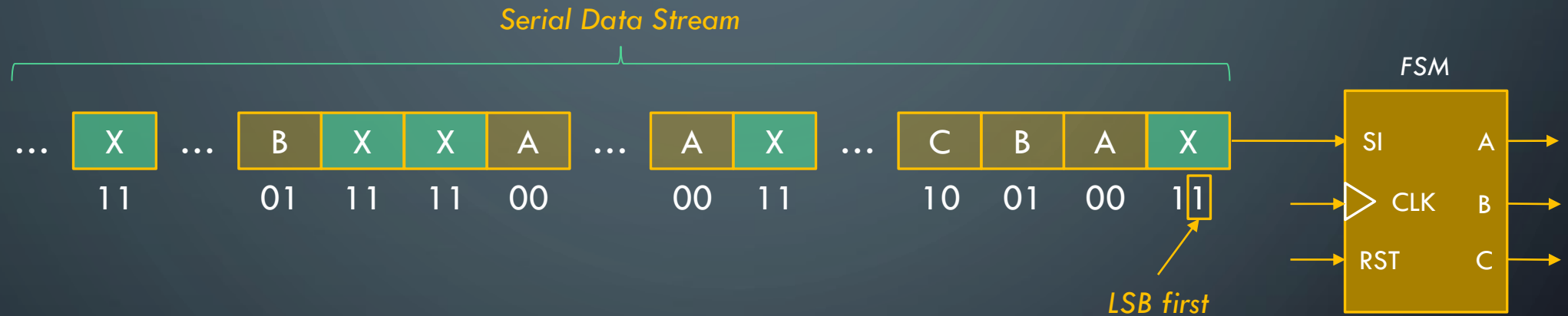
State Diagram



ПОЦП :: Л9 :: Язык VHDL :: последовательностные схемы :: FSM

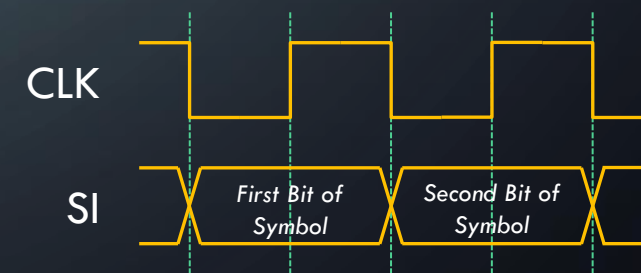
Example of FSM Design

Task: Design a digital finite state machine that decodes the characters A (00), B (01), and C (10) from a serial data stream. The symbol X (11) is a dummy symbol and is not to be decoded. All symbols can occur in any order.



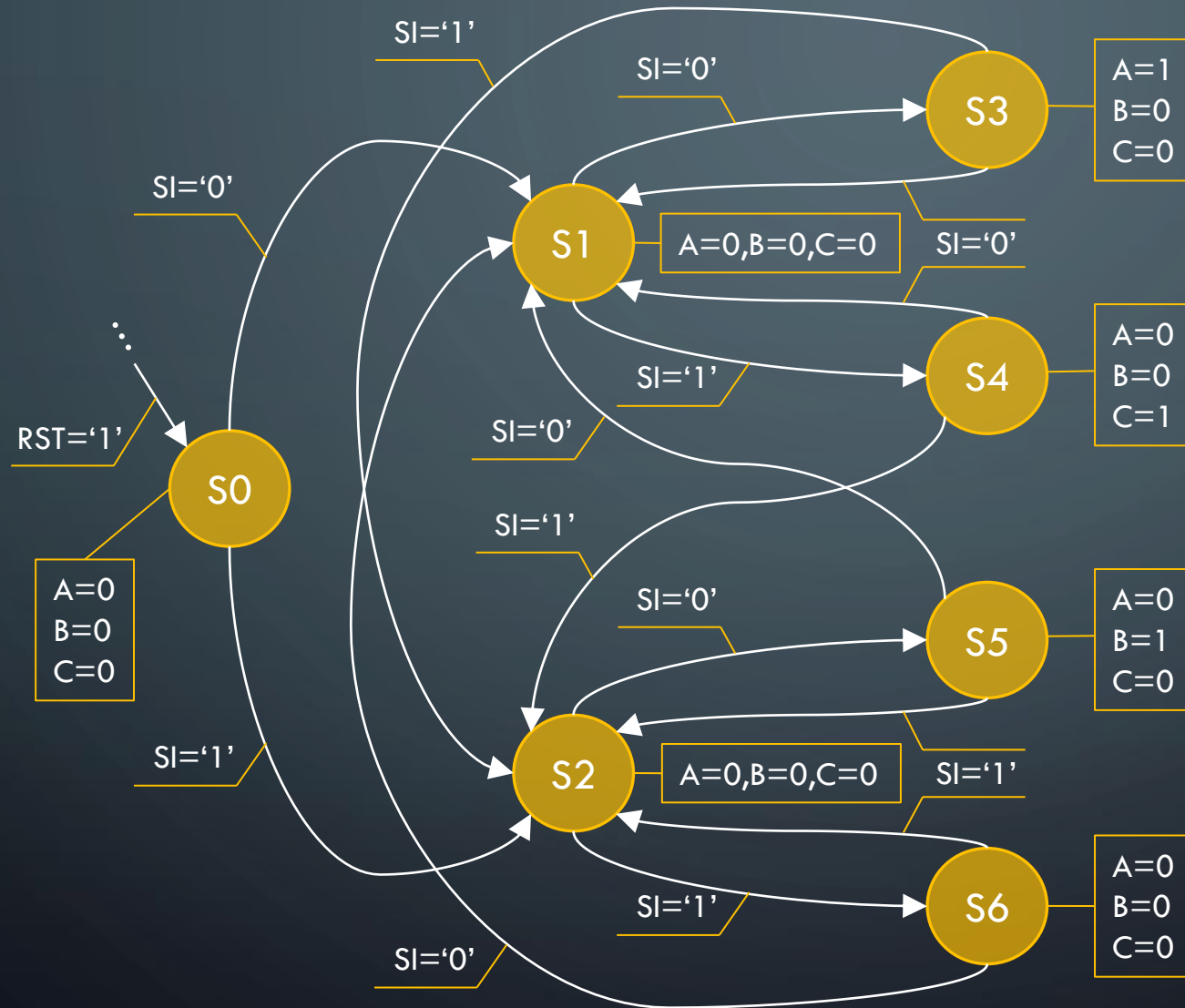
Port	Comment
SI	Serial data input
CLK	Clock input, active on the rising edge
RST	Synchronous reset input, active high
A	Goes high when the symbol A (00) is received
B	Goes high when the symbol B (01) is received
C	Goes high when the symbol C (10) is received

Symbol	Coding
A	00
B	01
C	10
X	11

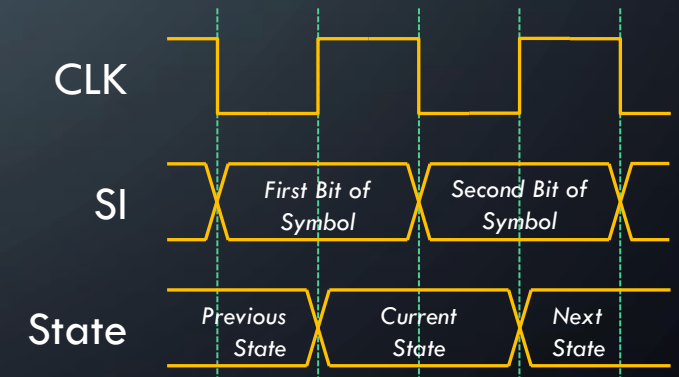


State Diagram

Example of State Diagram



State	Comment
S0	Initial state after reset
S1	Receive the LSB='0'
S2	Receive the LSB='1'
S3	Receive the MSB='0' (symbol A)
S4	Receive the MSB='1' (symbol C)
S5	Receive the MSB='0' (symbol B)
S6	Receive the MSB='1' (dummy symbol)



VHDL: FSM

```

Entity FSM is
  Port ( RST : in  std_logic;
        CLK  : in  std_logic;
        SI   : in  std_logic;
        A    : out std_logic;
        B    : out std_logic;
        C    : out std_logic );
End FSM;
Architecture Behavioral of FSM is
  type t_states is ( S0, S1, S2, S3, S4, S5, S6 );
  signal cur_state, nxt_state : t_states;
Begin
  FSM_MEM: process ( CLK, RST, nxt_state )
  begin
    if rising_edge( CLK ) then
      if RST = '1' then
        cur_state <= S0;
      else
        cur_state <= nxt_state;
      end if;
    end if;
  end process FSM_MEM;

```

VHDL: FSM

```

FSM_Delta: process ( cur_state, SI )
begin
  case cur_state is
    when S0 => if SI = '0' then
                  nxt_state <= S1;
                else
                  nxt_state <= S2;
                end if;
    when S1 => if SI = '0' then
                  nxt_state <= S3;
                else
                  nxt_state <= S4;
                end if;
    when S2 => if SI = '0' then
                  nxt_state <= S5;
                else
                  nxt_state <= S6;
                end if;
    when S3 | S4 | S5 | S6 =>
      if SI = '0' then
        nxt_state <= S1;
      else
        nxt_state <= S2;
      end if;
    when others => nxt_state <= S0;
  end case;
end process FSM_Delta;

```

VHDL: FSM

```

FSM_Lambda_Stop: process ( cur_state )
begin
  case cur_state is
    when S3 => A <= '1';
              B <= '0';
              C <= '0';
    when S5 => A <= '0';
              B <= '1';
              C <= '0';
    when S4 => A <= '0';
              B <= '0';
              C <= '1';
    when others =>
      A <= '0';
      B <= '0';
      C <= '0';
  end case;
end process;
End Behavioral;

```

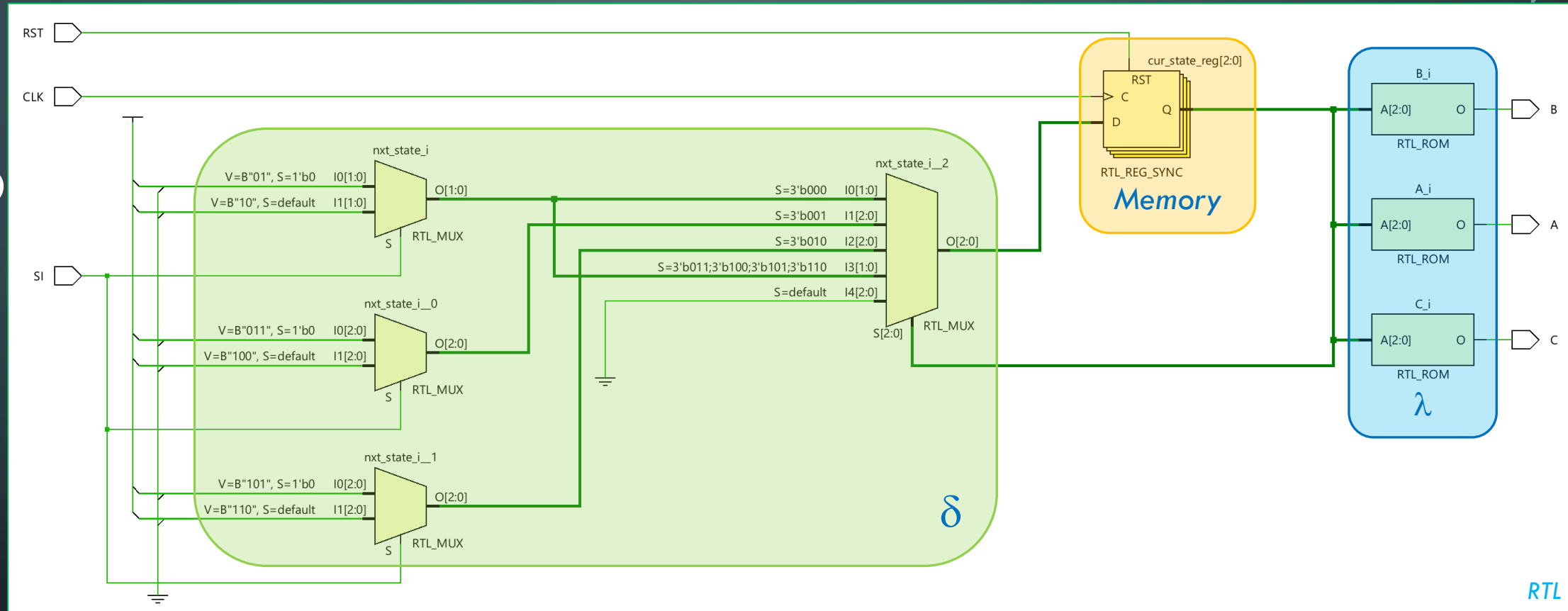
Moore FSM

Three processes:
Synchronous memory,
Combinational Logic for next state,
Combinational Logic for outputs

ПОЦП :: Л9 :: Язык VHDL :: последовательные схемы :: FSM

RTL Scheme

FSM Design

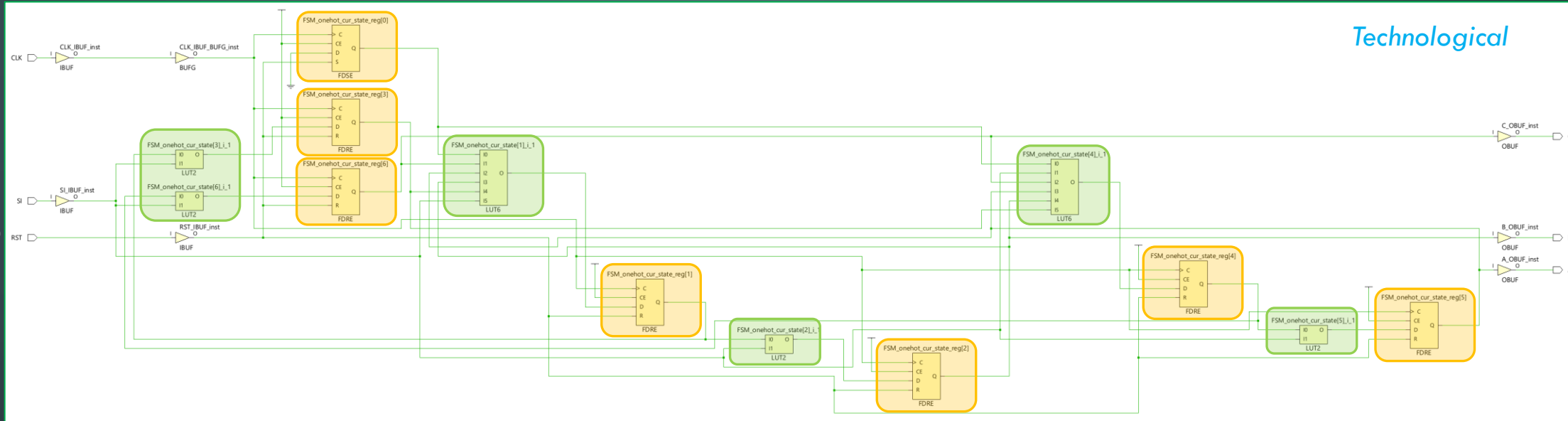


Moore FSM

Synchronous memory,
Combinational Logic for next state δ ,
Combinational Logic for outputs λ

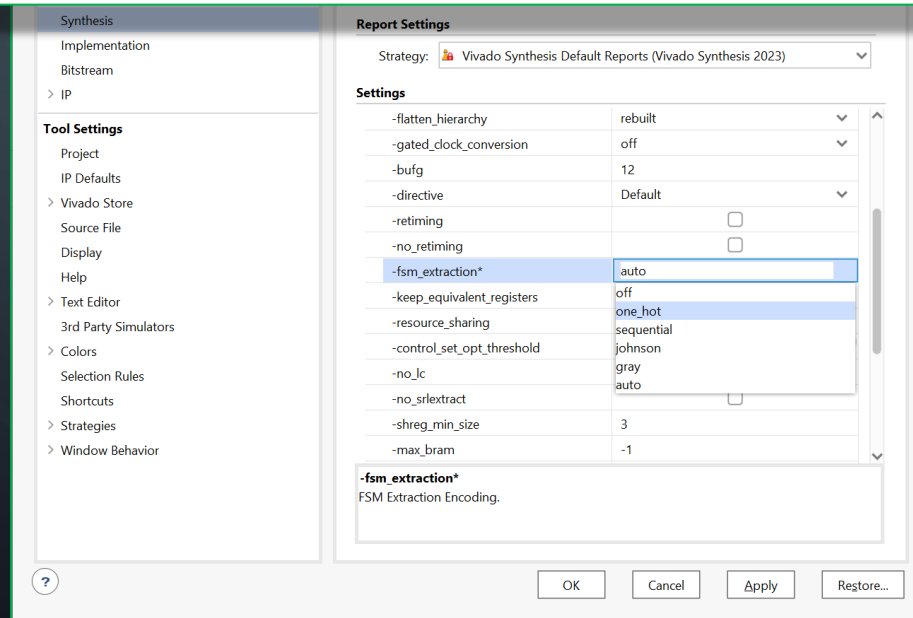
FSM Design

Technological



- Memory
- δ Logic
- λ Logic

Medvedev FSM

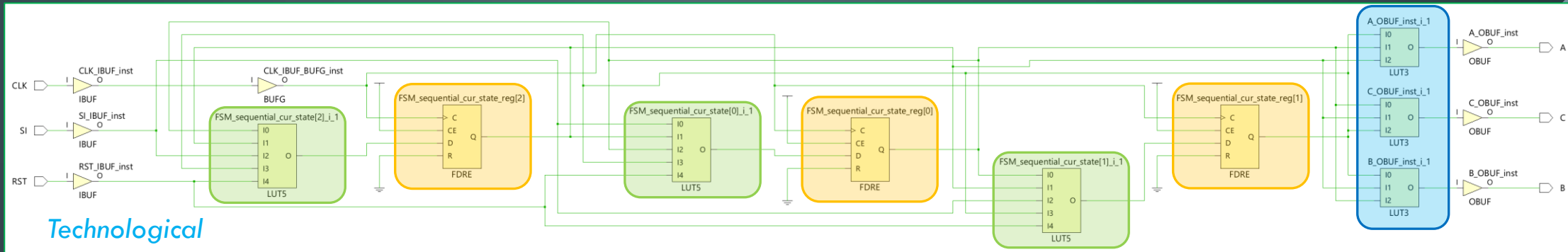


```
...
type t_states is ( S0, S1, S2, S3, S4, S5, S6 );
signal cur_state, nxt_state : t_states;
```

S0	0000001
S1	0000010
S2	0000100
S3	0001000
S4	0010000
S5	0100000
S6	1000000

ПОЦП :: Л9 :: Язык VHDL :: последовательностные схемы :: FSM

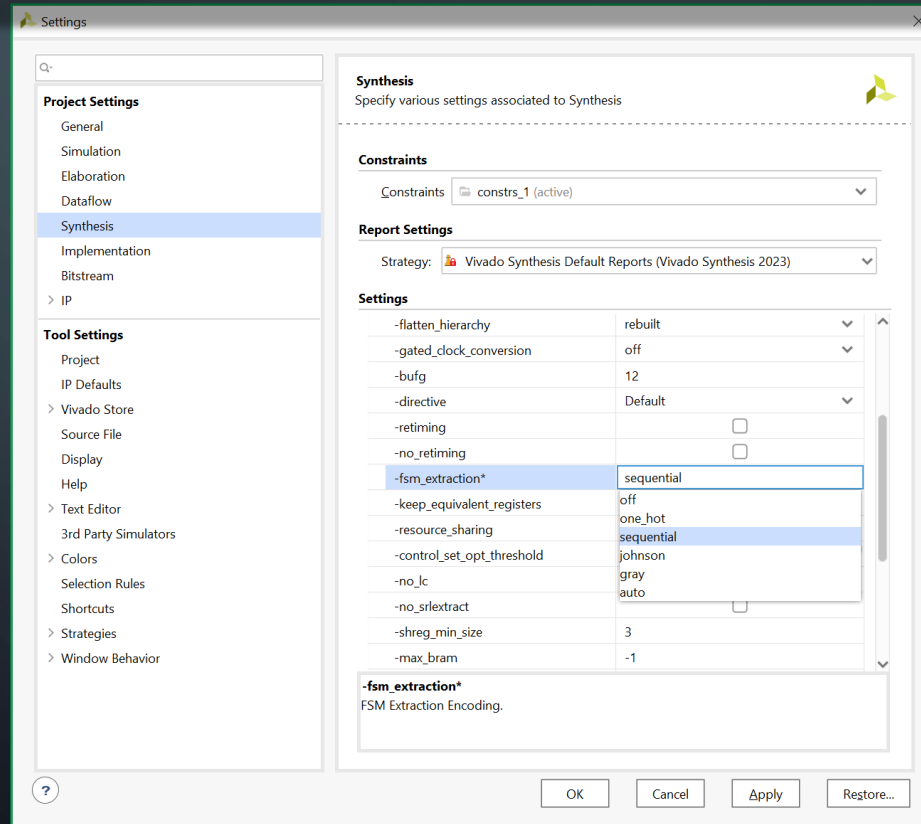
FSM States Encoding



FSM Design

- Memory
- δ Logic
- λ Logic

**Moore
FSM**



...

```
type t_states is ( S0, S1, S2, S3, S4, S5, S6 );
signal cur_state, nxt_state : t_states;
```

...

Sequential

S0	001
S1	010
S2	011
S3	100
S4	101
S5	110
S6	111

Testbench for FSM**VHDL: Package and Function**

```

Library ieee;
Use ieee.std_logic_1164.all;

Package MYPKG is
    function str_to_slv( str : string ) return std_logic_vector;
End package;

```

X	C	B	A	X
11	10	01	00	11

```

Library work;
Use work.MYPKG.all;

```

```

S : string := "XCBAX";
signal sd : std_logic_vector( 2*S'length-1 downto 0 ) := str_to_slv( S );
...
sd = "1110010011";

```

VHDL: Package and Function

```

Package body MYPKG is
    function str_to_slv( str : string ) return std_logic_vector is
        constant len : integer := str'length;
        variable res : std_logic_vector( 2*len-1 downto 0 );
        variable pos : integer;
    begin
        if len = 0 then
            return "";
        else
            pos := 2*len-1;
            for i in str'range loop
                case str( i ) is
                    when 'A' => res( pos downto pos-1 ) := "00";
                    when 'B' => res( pos downto pos-1 ) := "01";
                    when 'C' => res( pos downto pos-1 ) := "10";
                    when others =>
                        res( pos downto pos-1 ) := "11";
                end case;
                pos := pos - 2;
            end loop;
            return res;
        end if;
    end function;
End package body;

```

Testbench for FSM**VHDL: TestBench**

```

Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;
Library work;
Use work.MYPKG.ALL;
Entity TB_FSM is
    Generic ( TSEQ : string := "XBACBCACBAX" );
End TB_FSM;

Architecture Behavioral of TB_FSM is
    -- UUT declaration
    component FSM is
        Port ( RST, CLK, SI : in std_logic;
              A, B, C      : out std_logic );
    end component;
    -- Signals under test
    signal CLK : std_logic := '0';
    signal RST, SI, A, B, C : std_logic;
    -- Reference bit stream
    signal sref : std_logic_vector( 2*TSEQ'length-1 downto 0 ) := str_to_slv( TSEQ );
    -- Real bit stream
    signal sdat : std_logic_vector( sref'range );
    -- Enable signal for Monitor
    signal enable : std_logic := '0';

```

VHDL: TestBench

```

-- Time constants for CLK generation
constant HPeriod: time := 50 ns;
constant Period: time := 2*HPeriod;
-- Length of the test sequence in bits
constant N : integer := 2*TSEQ'length;

Begin
    -- Clock generation
    CLK <= not CLK after HPeriod;
    -- UUT instantiation
    UUT: FSM port map ( RST, CLK, SI, A, B, C );

```

VHDL: TestBench

```

Main: process
begin
    RST <= '0'; wait for Period;
    RST <= '1'; wait for Period;
    RST <= '0'; wait for Period;
    wait for N*Period;
    assert sdat = sref report "Error of decoding!" severity error;
    report "End of Simulation" severity failure;
end process Main;

enable <= '1' when falling_edge( RST );

PTPG: process ( CLK, enable, sref )
variable ptr : integer range sref'range;
begin
    if enable = '1' then
        if falling_edge( CLK ) then
            ptr := ptr + 1;
            if ptr = sref'length-1 then
                ptr := 0;
            end if;
        end if;
    else
        ptr := 0;
    end if;
    SI <= sref( ptr );
end process PTPG;

```

VHDL: TestBench

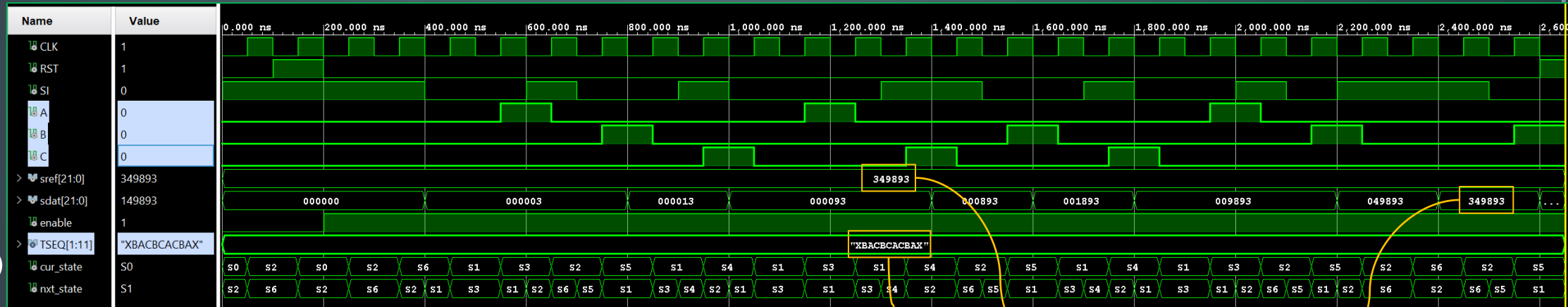
```

PMonitor: process ( CLK, enable, A, B, C )
variable tflag : boolean;
variable pos : integer;
variable abc : std_logic_vector( 2 downto 0 );
begin
    if enable = '1' then
        if falling_edge( CLK ) then
            abc := A & B & C;
            tflag := not tflag;
            if not( tflag ) then
                case abc is
                    when "100" => sdat( pos downto pos-1 ) <= "00";
                    when "010" => sdat( pos downto pos-1 ) <= "01";
                    when "001" => sdat( pos downto pos-1 ) <= "10";
                    when others => sdat( pos downto pos-1 ) <= "11";
                end case;
            end if;
            if pos /= sdat'high then
                pos := pos + 2;
            end if;
        end if;
    else
        tflag := false;
        pos := 1;
        sdat <= ( others => '0' );
    end if;
end process PMonitor;
End Behavioral;

```

ПОЦП :: Л9 :: Язык VHDL :: последовательностные схемы :: FSM

FSM Design



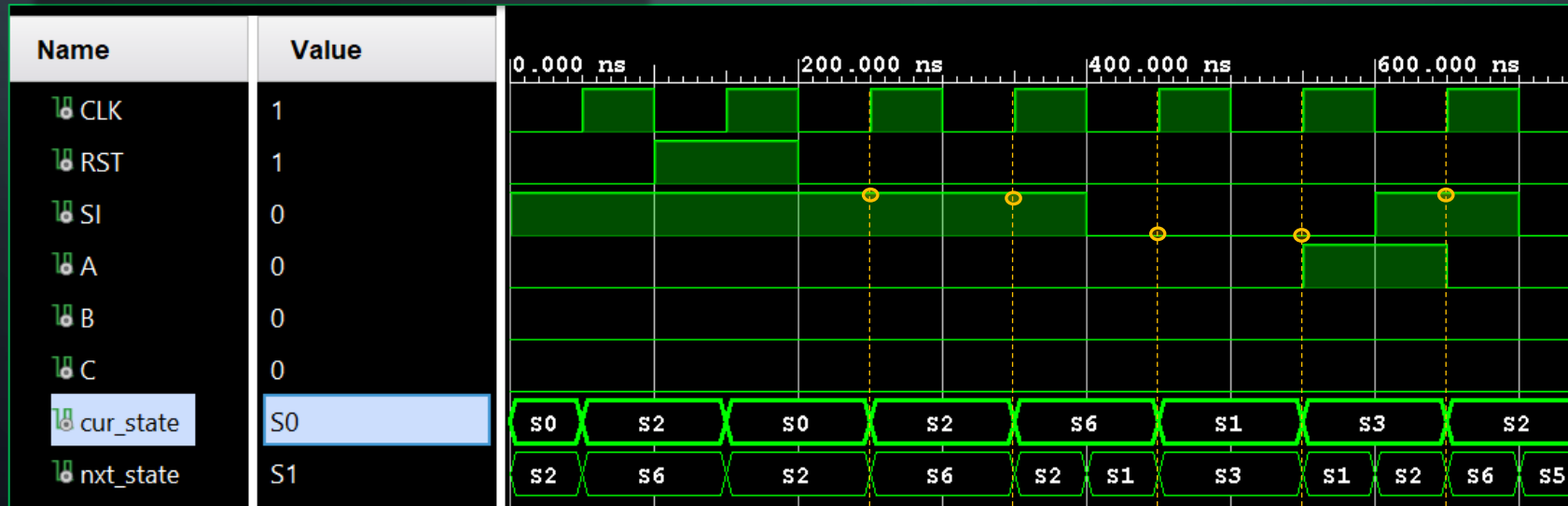
```
run all
Failure: End of Simulation
Time: 2500 ns Iteration: 0
$finish called at time : 2500
```

"XBACBCACBAX"

string

349893

HEX std_logic_vector



ПОЦП :: Л9 :: Язык VHDL :: последовательные схемы :: FSM

```
type t_states is ( S0, S1, S2, S3, S4, S5, S6 );  
signal cur_state, nxt_state : t_states;
```

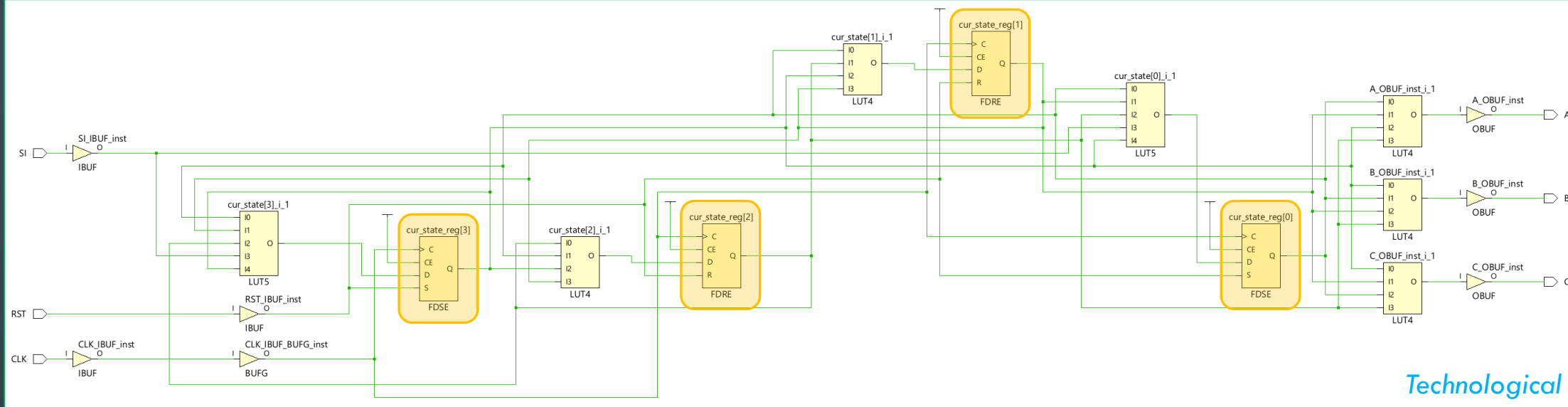
```
type t_states is ( S0, S1, S2, S3, S4, S5, S6 );  
signal cur_state, nxt_state : t_states;  
attribute fsm_encoding : string;  
attribute fsm_encoding of cur_state : signal is "sequential";
```

The legal values for this are **one_hot**, **sequential**, **johnson**, **gray**, **user_encoding**, and **none**. The **auto** value is the default, and allows the tool to determine best encoding. The **user_encoding** value tells the tool to infer a state machine, and use the encoding that you provide in the RTL.

VHDL: FSM

```
subtype t_states is std_logic_vector( 3 downto 0 );  
constant S0 : t_states := "1001";  
constant S1 : t_states := "1010";  
constant S2 : t_states := "0011";  
constant S3 : t_states := "1100";  
constant S4 : t_states := "0101";  
constant S5 : t_states := "0110";  
constant S6 : t_states := "1111";  
signal cur_state, nxt_state : t_states;  
attribute fsm_encoding : string;  
attribute fsm_encoding of cur_state : signal is "user_encoding";
```



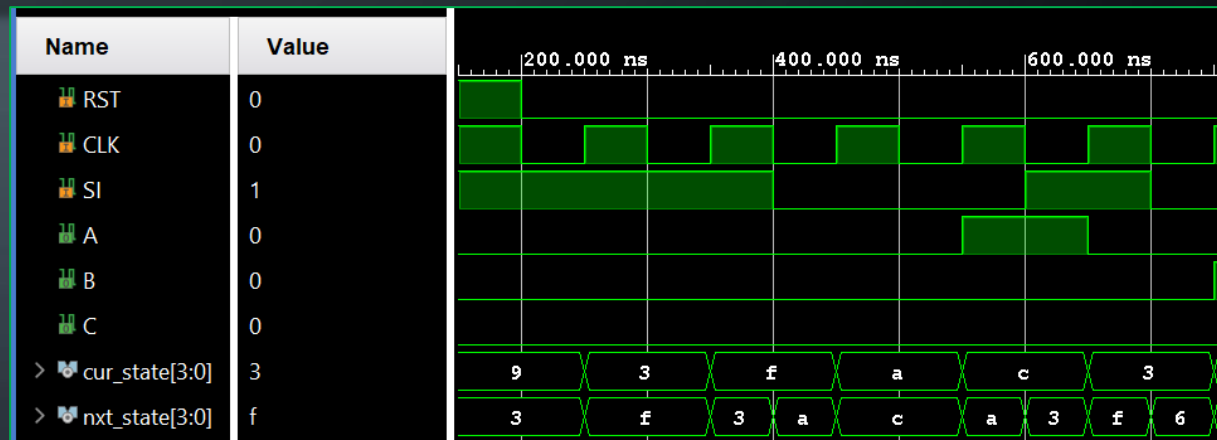


Technological

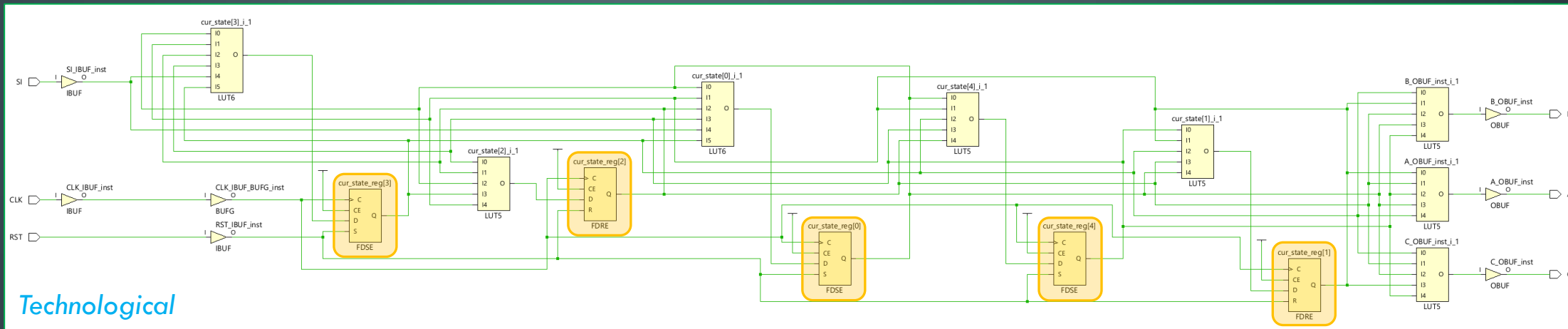
VHDL: FSM

```

subtype t_states is std_logic_vector( 3 downto 0 );
constant S0 : t_states := "1001";
constant S1 : t_states := "1010";
constant S2 : t_states := "0011";
constant S3 : t_states := "1100";
constant S4 : t_states := "0101";
constant S5 : t_states := "0110";
constant S6 : t_states := "1111";
signal cur_state, nxt_state : t_states;
attribute fsm_encoding : string;
attribute fsm_encoding of cur_state : signal is "user_encoding";
    
```



Not very convenient

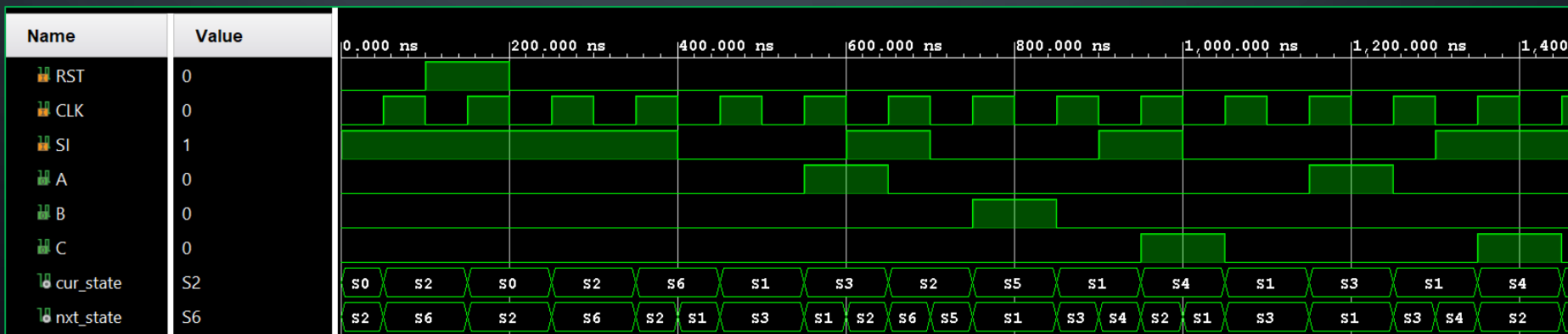


VHDL: FSM

```

type t_states is ( S0, S1, S2, S3, S4, S5, S6 );
attribute enum_encoding : string;
attribute enum_encoding of t_states : type is "11001 11010 10011 11100 10101 10110 11111";
signal cur_state, nxt_state : t_states;
attribute fsm_encoding : string;
attribute fsm_encoding of cur_state : signal is "user_encoding";

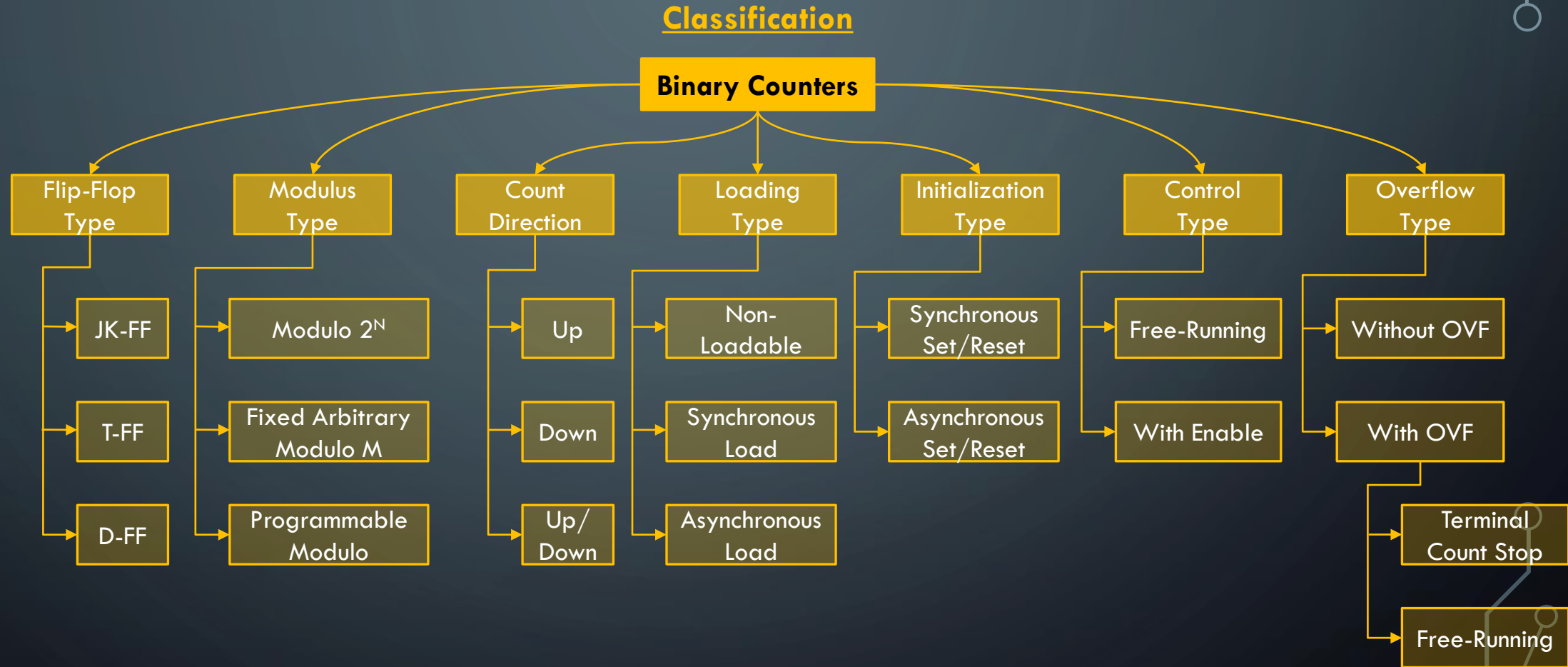
```



ПОЦП :: Л9 :: Язык VHDL :: последовательные схемы :: FSM

Binary counters are digital sequential circuits, often built using flip-flops, that count clock pulses in binary code. Any type of binary counters is a Finite State Machine.

Binary Counter



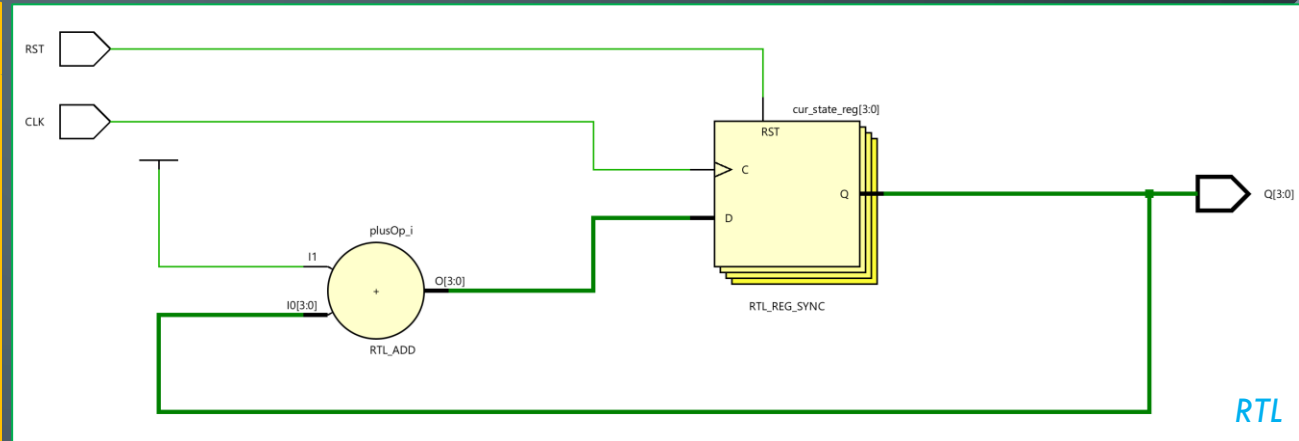
Example of a Binary Counter

VHDL: Binary Counter #1

```

Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;
Use IEEE.STD_LOGIC_ARITH.ALL;
Use IEEE.STD_LOGIC_UNSIGNED.ALL;
Entity BCNT_1 is
    Generic ( N : integer range 1 to 32 := 4 );
    Port ( RST : in std_logic;
          CLK : in std_logic;
          Q : out std_logic_vector ( N-1 downto 0 ) );
End BCNT_1;
Architecture Behavioral of BCNT_1 is
    signal cur_state, nxt_state : std_logic_vector ( N-1 downto 0 );
Begin
    CNT_MEM: process ( RST, CLK, nxt_state )
    begin
        if rising_edge( CLK ) then
            if RST = '1' then
                cur_state <= ( others => '0' );
            else
                cur_state <= nxt_state;
            end if;
        end if;
    end process CNT_MEM;
    CNT_Delta: nxt_state <= cur_state + '1';
    CNT_Lambda: Q <= cur_state;
End Behavioral;

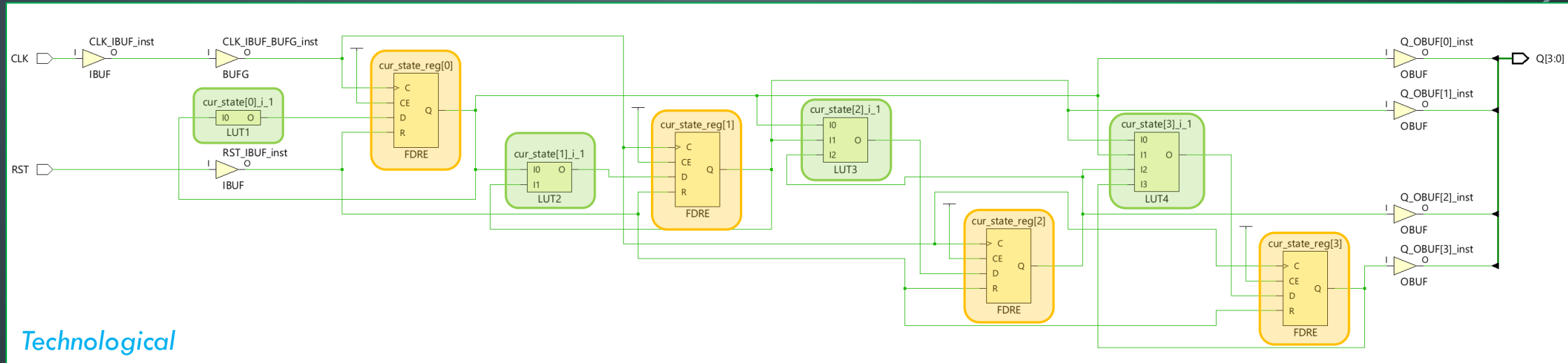
```



Autonomous FSM

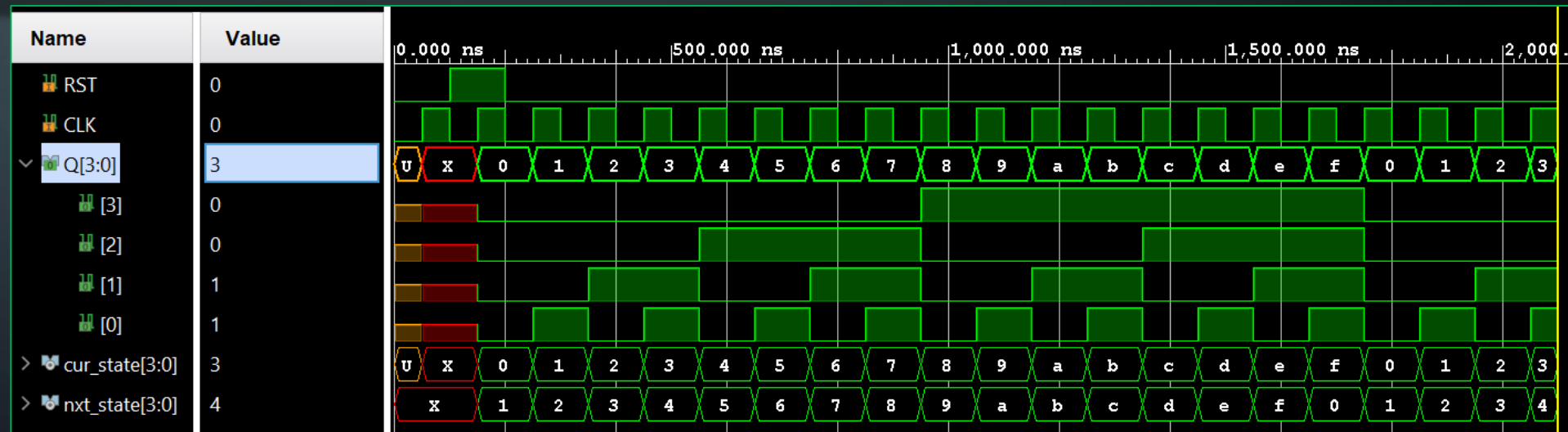
Flip-Flop Type	D-FF	Initialization Type	Synchronous Reset
Modulus Type	Modulo 2^N	Control Type	Free-Running
Count Direction	Up	Overflow Type	Without OVF
Loading Type	Non-Loadable		

Example of a Binary Counter



Binary Counter

- Memory
 - δ Logic
 - λ Logic
- Autonomous FSM**



Binary Counter

ПОЦП :: Л9 :: Язык VHDL :: последовательностные схемы :: FSM

Example of a Binary Counter

VHDL: Binary Counter #2

```
Entity BCNT_2 is
  Generic ( N : integer range 1 to 32 := 4 );
  Port ( RST, LD, EN : in std_logic;
        CLK : in std_logic;
        D : in std_logic_vector ( N-1 downto 0 );
        Q : out std_logic_vector ( N-1 downto 0 ) );
End BCNT_2;
Architecture Behavioral of BCNT_2 is
  signal cur_state, nxt_state : std_logic_vector ( N-1 downto 0 );
Begin
  CNT_MEM: process ( RST, CLK, nxt_state )
  begin
    if rising_edge( CLK, EN, nxt_state ) then
      if RST = '1' then
        cur_state <= ( others => '0' );
      elsif EN = '1' then
        cur_state <= nxt_state;
      end if;
    end if;
  end process CNT_MEM;
```

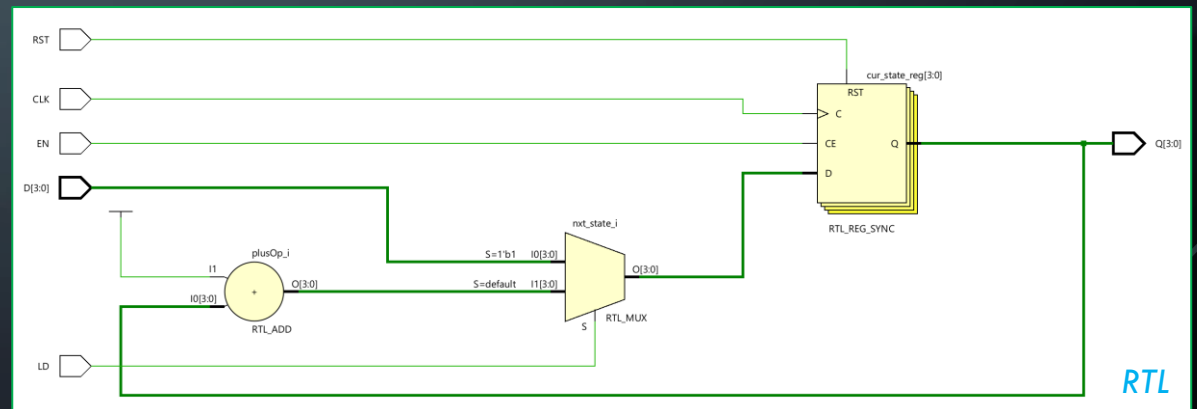
VHDL: Binary Counter #2

```
CNT_Delta: process ( LD, D, cur_state )
begin
  if LD = '1' then
    nxt_state <= D;
  else
    nxt_state <= cur_state + '1';
  end if;
end process CNT_Delta;

CNT_Lambda: Q <= cur_state;
End Behavioral;
```

Medvedev FSM

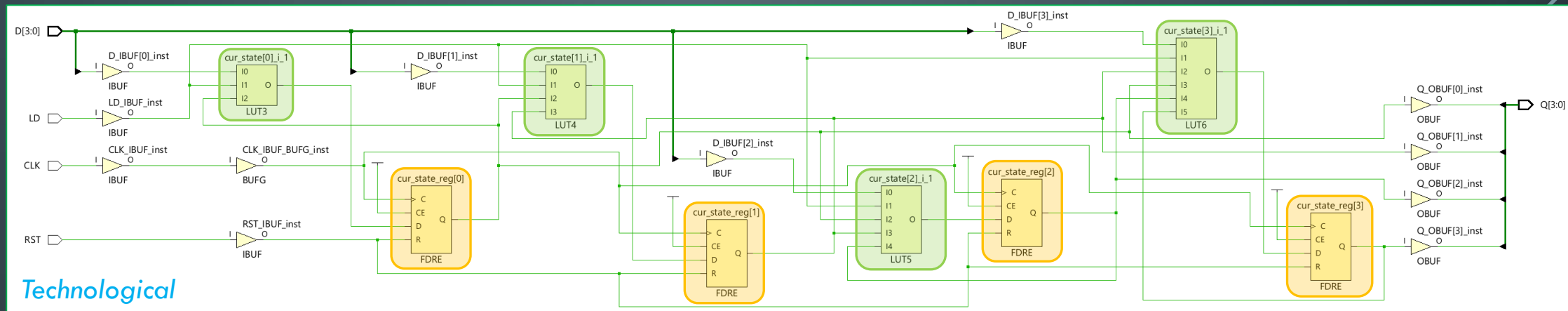
Flip-Flop Type	D-FF
Modulus Type	Modulo 2^N
Count Direction	Up
Loading Type	Synchronous Load
Initialization Type	Synchronous Reset
Control Type	With Enable
Overflow Type	Without OVF



RTL

ПОЦП :: Л9 :: Язык VHDL :: последовательностные схемы :: FSM

Example of a Binary Counter

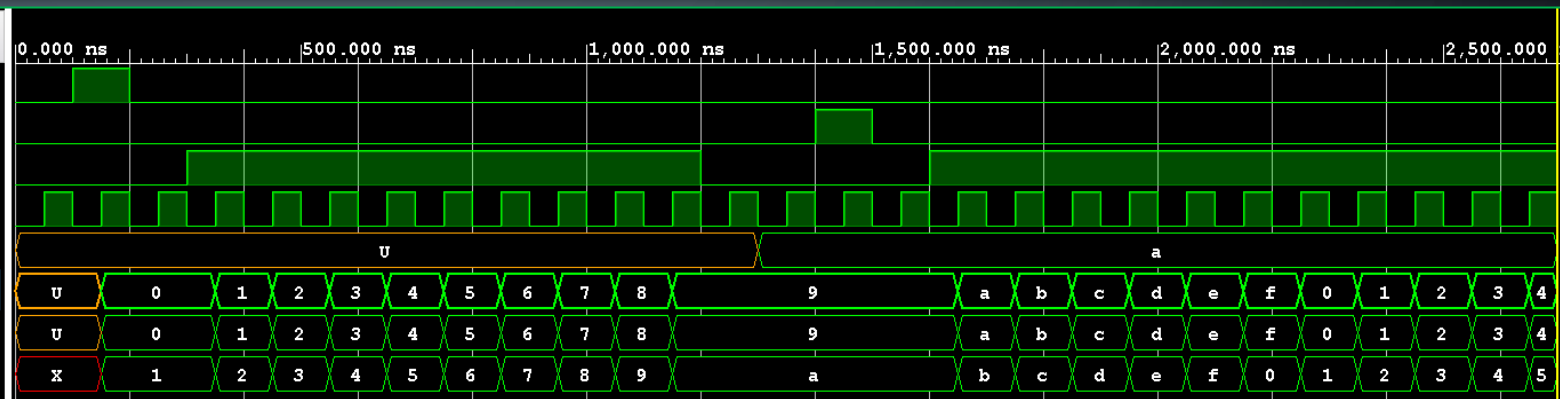


Technological

- Memory
- δ Logic
- λ Logic

Medvedev
FSM

Name	Value
RST	0
LD	0
EN	1
CLK	0
D[3:0]	a
Q[3:0]	4
cur_state[3:0]	4
nxt_state[3:0]	5



D-FF

Modulo 2^N

Up

Non-
Loadable

Synchronous
Load

With Enable

Without OVF

Binary Counter

ПОЦП :: Л9 :: Язык VHDL :: последовательностные схемы :: FSM

Example of a Binary Counter

VHDL: Binary Counter #3

```
Entity BCNT_3 is
  Generic ( N : integer range 1 to 32 := 4 );
  Port ( RST, LD, EN : in std_logic;
        CLK : in std_logic;
        OVF : out std_logic;
        D : in std_logic_vector ( N-1 downto 0 );
        Q : out std_logic_vector ( N-1 downto 0 ) );
End BCNT_3;
```

Architecture Behavioral of BCNT_3 is

```
signal s_ovf : std_logic;
signal cur_state, nxt_state : std_logic_vector ( N downto 0 );
```

Begin

```
CNT_MEM: process ( RST, CLK, EN, nxt_state, s_ovf )
```

```
begin
```

```
  if rising_edge( CLK ) then
```

```
    if RST = '1' then
```

```
      cur_state <= ( others => '0' );
```

```
    elsif EN = '1' and s_ovf = '0' then
```

```
      cur_state <= nxt_state;
```

```
    end if;
```

```
  end if;
```

```
end process CNT_MEM;
```

VHDL: Binary Counter #3

```
CNT_Delta: process ( LD, D, cur_state )
```

```
begin
```

```
  if LD = '1' then
```

```
    nxt_state <= '0' & D;
```

```
  else
```

```
    nxt_state <= cur_state + '1';
```

```
  end if;
```

```
end process CNT_Delta;
```

```
CNT_Lambda: Q <= cur_state( Q'range );
```

```
OVF_Lambda: s_ovf <= cur_state( N );
```

```
OVF <= s_ovf;
```

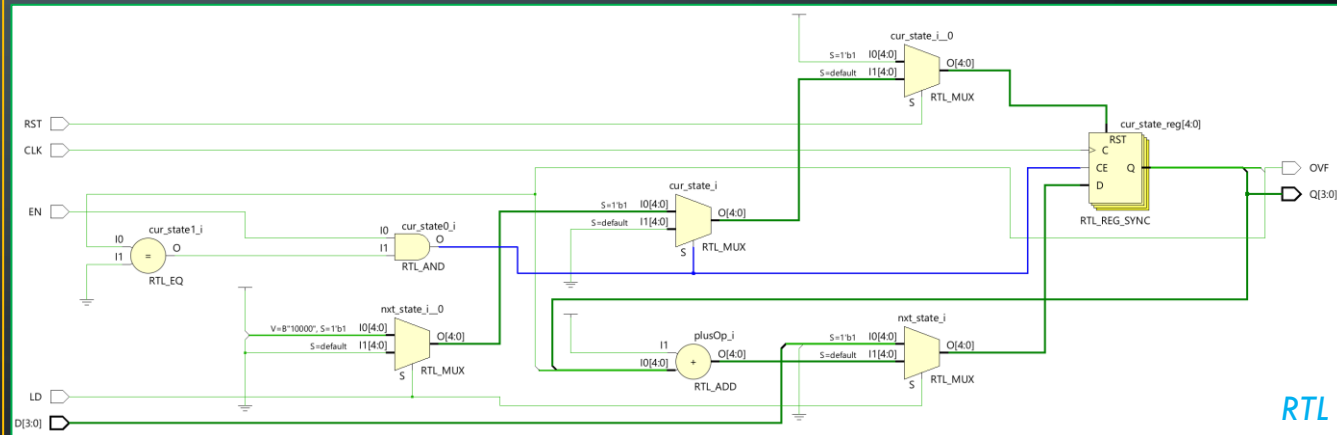
```
End Behavioral;
```

Overflow
Type

With OVF

Terminal
Count Stop

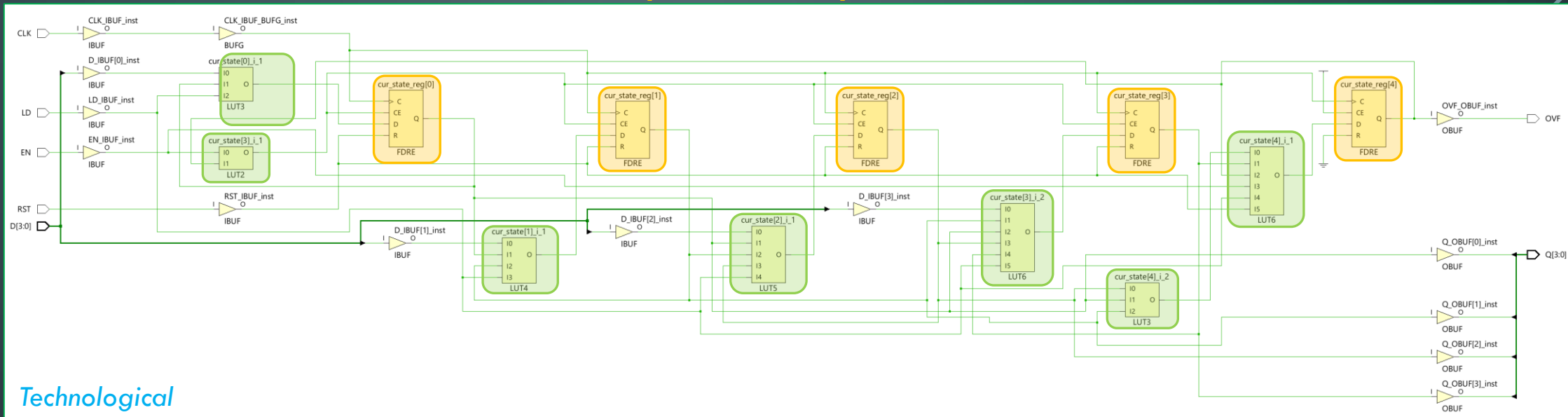
Overflow signal as
Clock Enable for DFFs



Medvedev FSM

ПОЦП :: Л9 :: Язык VHDL :: последовательностные схемы :: FSM

Example of a Binary Counter

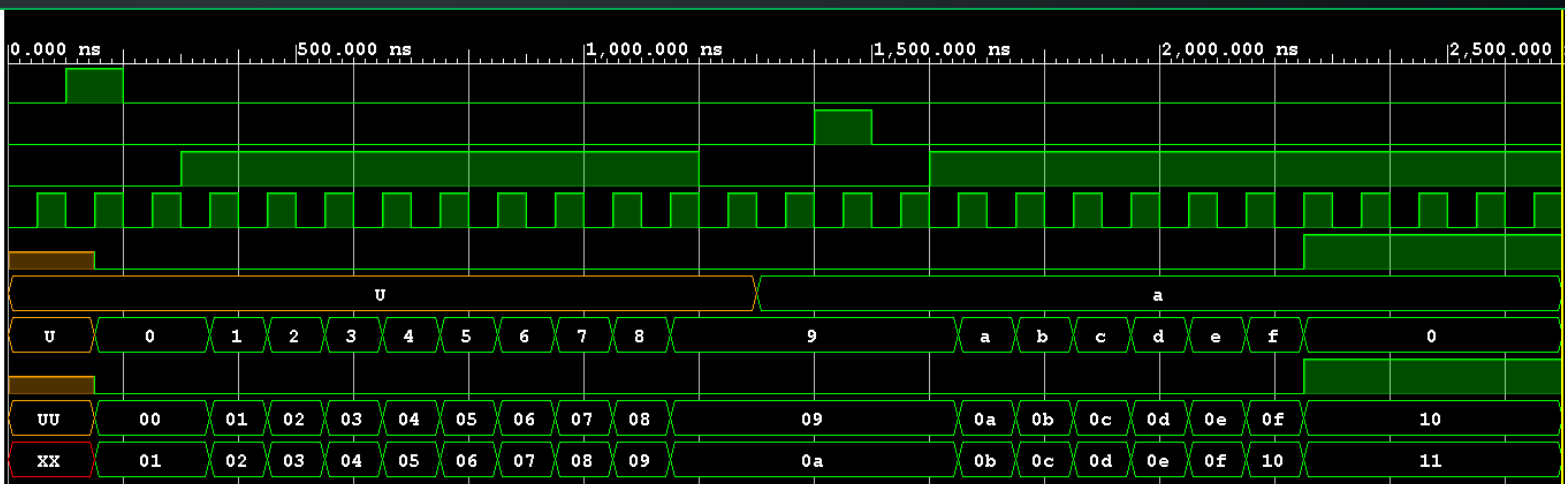


Technological

- Memory
- δ Logic
- λ Logic

Medvedev
FSM

Name	Value
RST	0
LD	0
EN	1
CLK	0
OVF	1
D[3:0]	a
Q[3:0]	0
s_ovf	1
cur_state[4:0]	10
nxt_state[4:0]	11



With OVF

Terminal
Count Stop

The background of the slide is a deep space image showing the cosmic web, with intricate filaments of gas and dark matter stretching across the frame. The filaments are illuminated in shades of blue, purple, and orange. A bright star with a four-pointed diffraction pattern is visible in the upper left. The entire scene is set against a dark, star-filled sky. In the corners, there are stylized white line-art overlays resembling electronic circuit boards or neural network connections.

Per Aspera Ad Astra